

Disciplina: Sistemas operacionais

Professor: Ary Henrique Moraes de Oliveira

Alunos: Daniel Nolêto Maciel Luz e João Victor Walcacer Giani

Código 1:

Este código implementa o problema do Produtor-Consumidor sem controle de concorrência, utilizando duas threads: uma para o produtor e outra para o consumidor. O buffer é um array de tamanho fixo ($N_ITENS = 10$), onde o produtor escreve e o consumidor lê. Os índices início e final controlam as operações de leitura e escrita. O produtor gera 30 itens e os coloca no buffer, atualizando o índice final de forma circular e fazendo uma pausa de 5 segundos. O consumidor lê 30 itens, atualiza o índice início circularmente e faz uma pausa de 1 segundo. O problema é que não há controle de concorrência, permitindo que ambos acessem o buffer simultaneamente, resultando em condições de corrida. A main inicializa o buffer, cria as threads e espera que ambas terminem.

Código 2:

Este código aborda o problema do Produtor-Consumidor com controle de concorrência através de espera ocupada. O buffer é um array de tamanho fixo ($N_ITENS = 10$) e os índices início e final controlam as operações. A variável `cont` rastreia a quantidade de itens no buffer, onde `cont == N_ITENS` indica que o buffer está cheio e `cont == 0` que está vazio. O produtor espera enquanto o buffer está cheio, escreve um item, atualiza final e incrementa `cont`, fazendo uma pausa aleatória. O consumidor espera enquanto o buffer está vazio, lê um item, atualiza início e decrementa `cont`, verificando erros. O problema é que a espera ocupada desperdiça recursos da CPU e a variável `cont` não é protegida, podendo causar inconsistências. O main inicializa o buffer, cria as threads e espera que ambas terminem.

Código 3:

Este código também implementa o problema do Produtor-Consumidor com controle de concorrência via espera ocupada, mas com uma abordagem diferente. O buffer é um array de tamanho fixo ($N_ITENS = 10$) e os índices início e final controlam as operações. A variável `cont` rastreia a quantidade de itens. O produtor espera enquanto o buffer está cheio, escreve um item, atualiza final e usa uma variável auxiliar para manipular `cont`. Após produzir todos os itens, imprime "Produção encerrada". O consumidor espera enquanto o buffer está vazio, lê um item, atualiza início e usa uma variável auxiliar para manipular `cont`, imprimindo "Consumo encerrado" ao final. O problema é a espera ocupada e as condições de corrida. O main inicializa o buffer, cria as threads e espera que ambas terminem.

Código 4:

Este código implementa o problema do Produtor-Consumidor utilizando semáforos para controle de concorrência. O buffer é um array de tamanho fixo ($N_ITENS = 30$) e os índices início e final controlam as operações. Dois semáforos são utilizados: `pos_vazia` controla as posições disponíveis e `pos_ocupada` controla as ocupadas. O produtor espera até haver uma posição vazia, escreve um item, atualiza final e incrementa `pos_ocupada`. O consumidor espera até haver um item disponível, lê um item, atualiza início e incrementa `pos_vazia`. O main inicializa os semáforos, cria as threads e espera que ambas terminem, destruindo os semáforos após a execução. As vantagens incluem a garantia de que o produtor e o consumidor não acessem o buffer simultaneamente e a eliminação da espera ocupada.